	NAME		
	REGISTRATION NUMBER		**MARKS**
	CLASS		/100
	SESSION	I 2026/2027	
	COURSE CODE	DFP40443-FULL STACK WEB DEVELOPMENT	
	ASSESSMENT	MINI PROJECT 2	
	LECTURER		

CLO 1 : Integrate PHP and MySQL with Bootstrap to build database-driven web applications projects. (P4, PLO3)

Topic : 4.1 User Authentication & Authorization
4.3 Form Processing and Validation
4.4 Multi-Page Application Development
5.1 AJAX Development
5.2 File Handling & Upload
5.3 Deployment & Version Control

Duration : 4 Hours

INSTRUCTION: Answer all the questions. This task is a group based with maximum of 4 members in a group.

QUESTION 1:

You are required to develop an **Student Portfolio & Final Year Project (FYP) Showcase System** using PHP programming language. The system acts as a centralized academic repository for Jabatan Teknologi Maklumat & Komunikasi, permitting students to showcase their software applications and reports, while allowing Lecturers/Admins to oversee submissions, manage categories, and inspect directories.

(CLO1, P4)

The system must include the following requirements:

A : User Authentication & Authorization

1. Build a registration and login system using PHP and MySQL.
2. Implement password hashing using password_hash() and verification using password_verify().
3. Assign user roles (Admin and Student).

4. Restrict access to pages based on user roles.
5. Manage sessions securely using $_SESSION.
6. Redirect users if session is not valid.
7. Expected page login and registration shown in Figure 1-2.

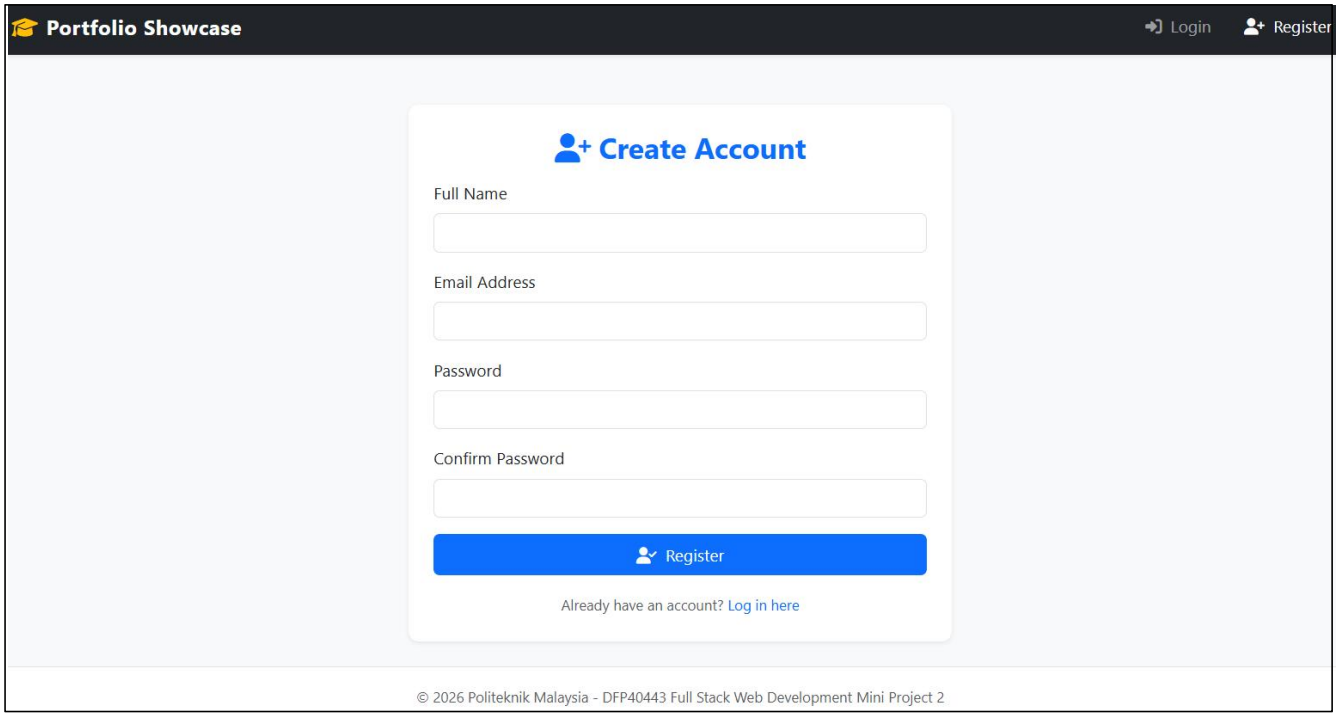


Figure 1

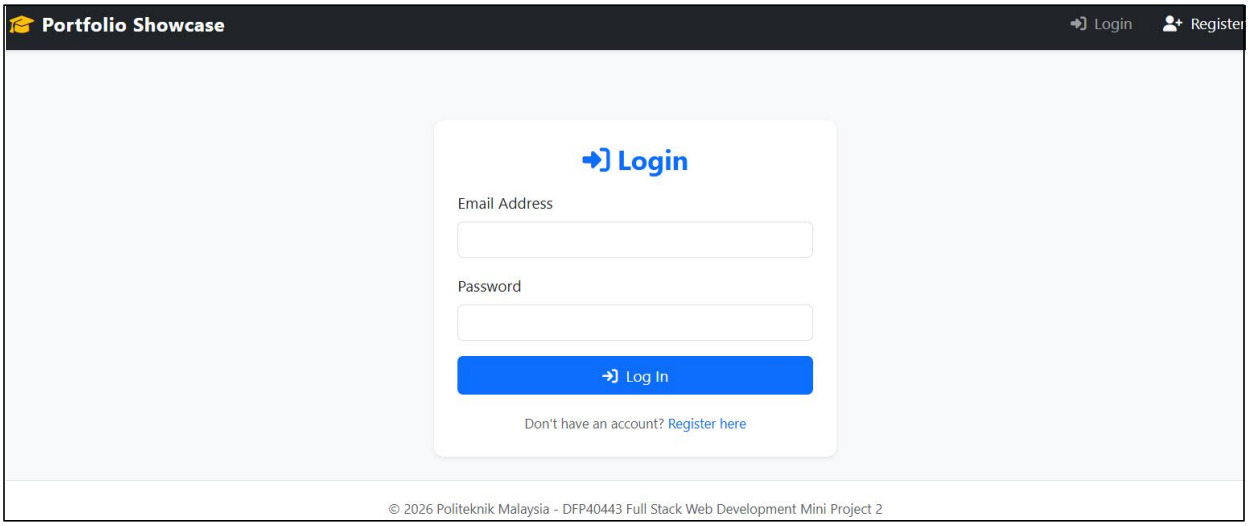


Figure 2

B : Form Processing and Validation

1. Implement form handling using POST method for all data submission.
2. Perform client-side validation using JavaScript on all input forms.
3. Perform server-side validation using PHP before processing or storing data.
4. Display appropriate error and success messages for all forms.
5. File upload must be implemented in the Student Submission page.
6. Validation must be applied in the following pages:
   a. Register Page (Student registration form validation)
   b. Login Page (Credential validation)
   c. Admin Page – Create Assignment (Input validation for assignment title and description).

   Expected page for student shown in Figure 3-5.

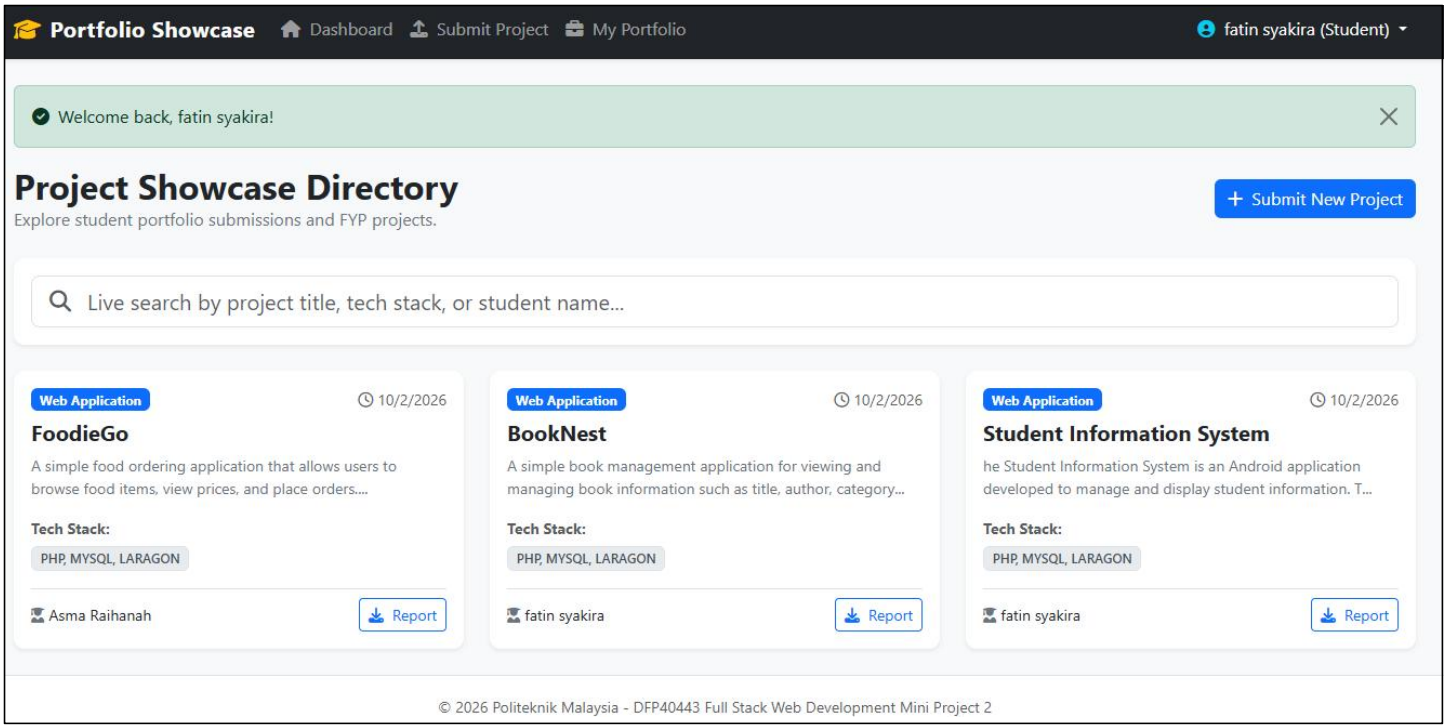


Figure 3

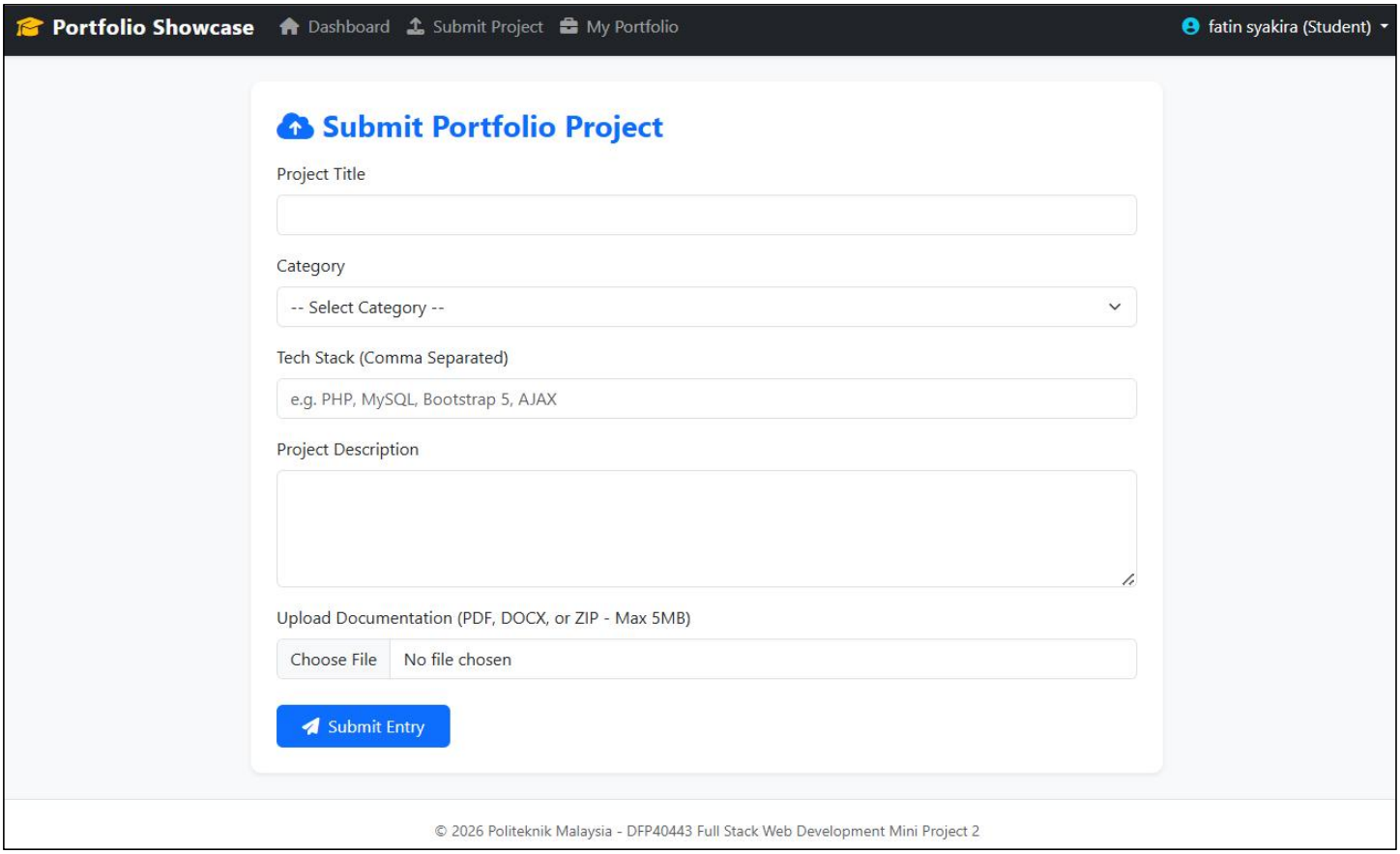


Figure 4

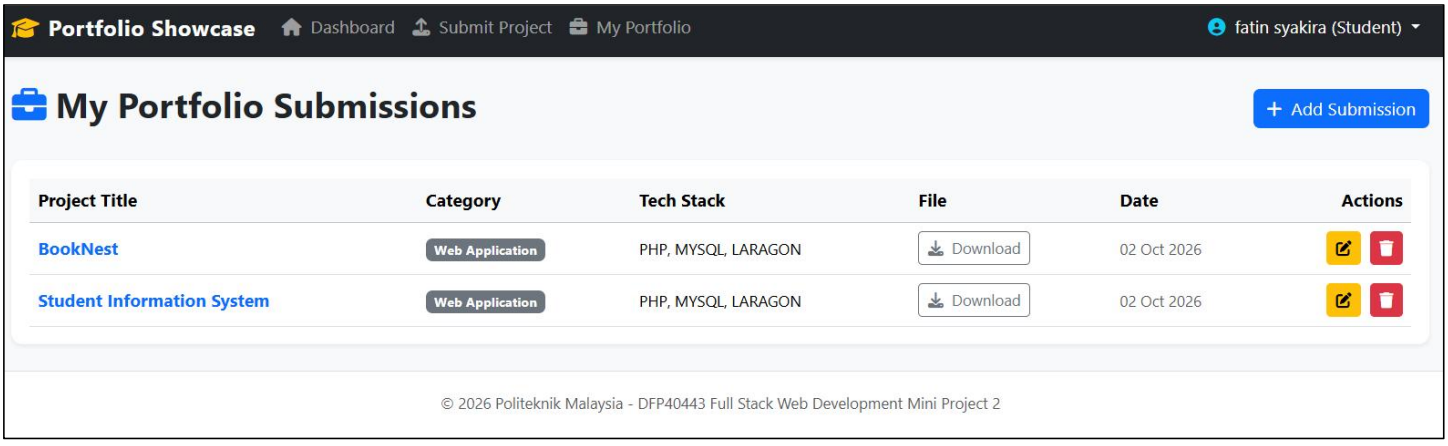


Figure 5

d. Student Page – Submit Assignment (File upload validation including file type and size).

Expected page for admin shown in Figure 6-8.

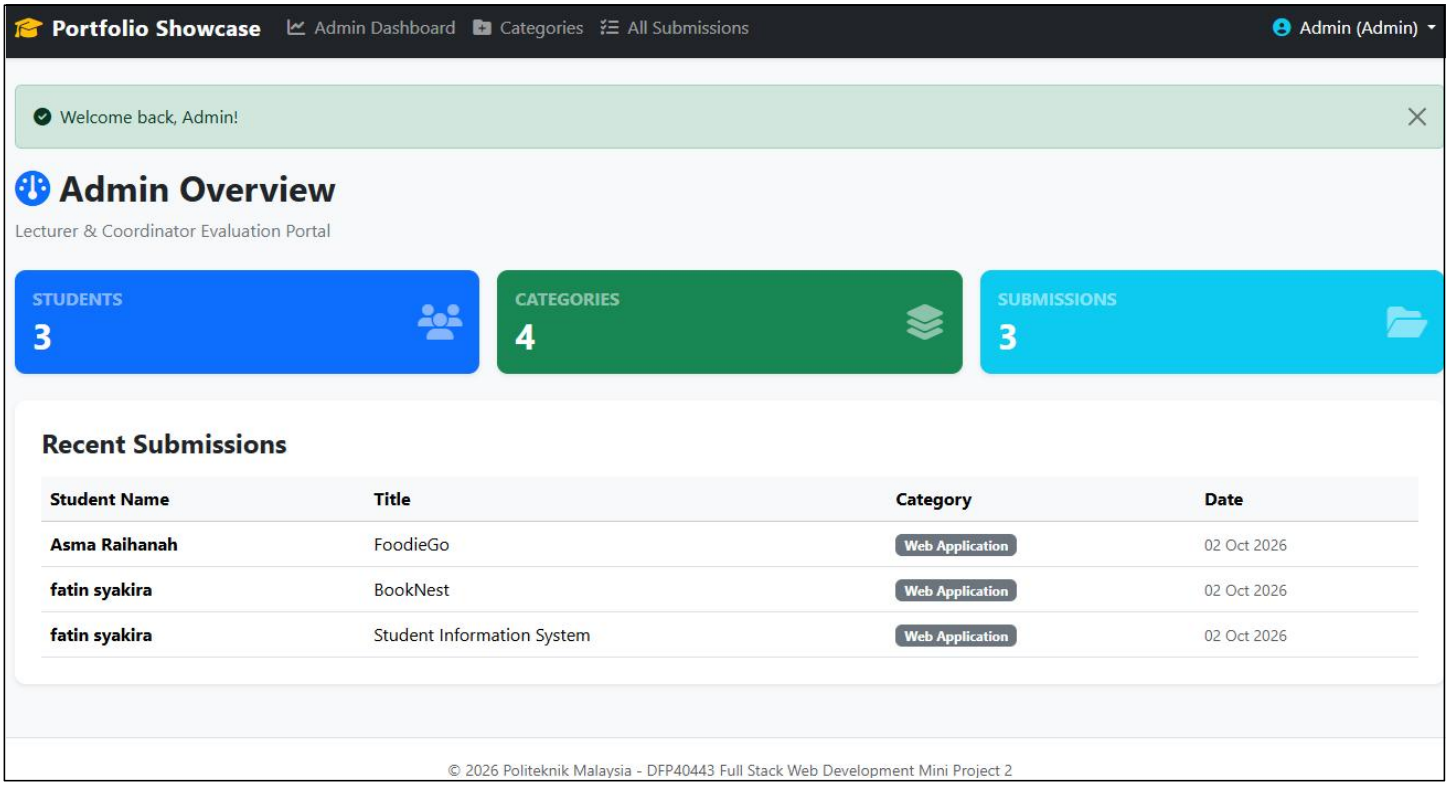


Figure 6

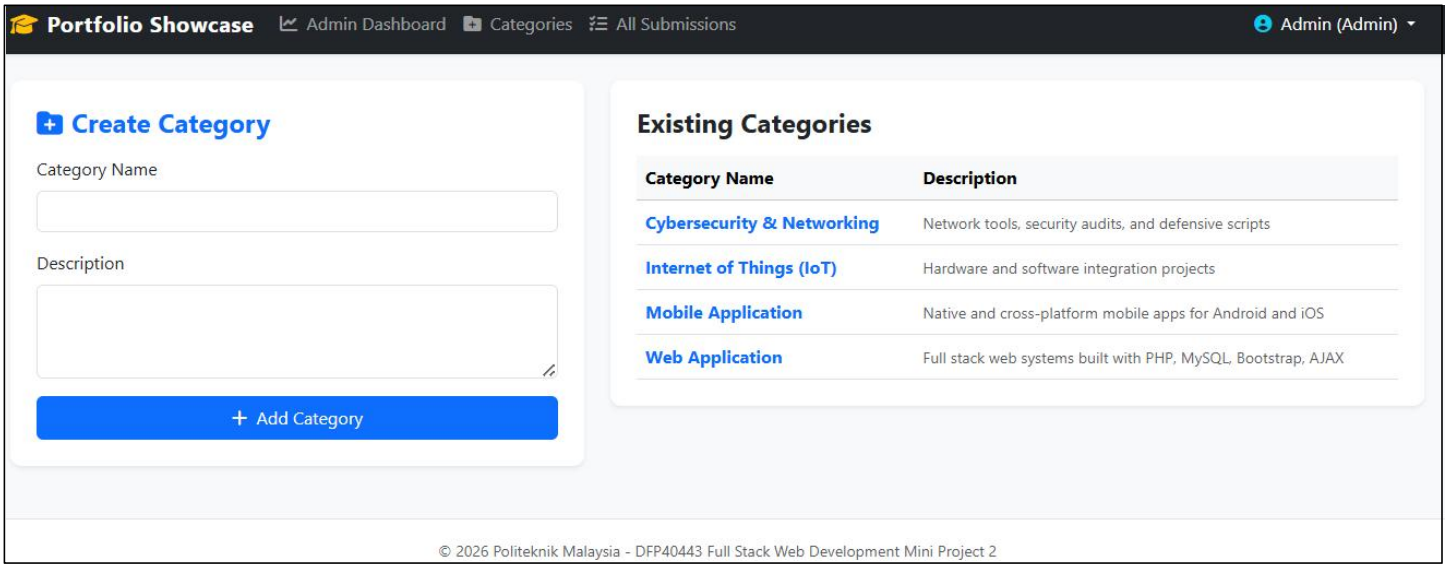


Figure 7

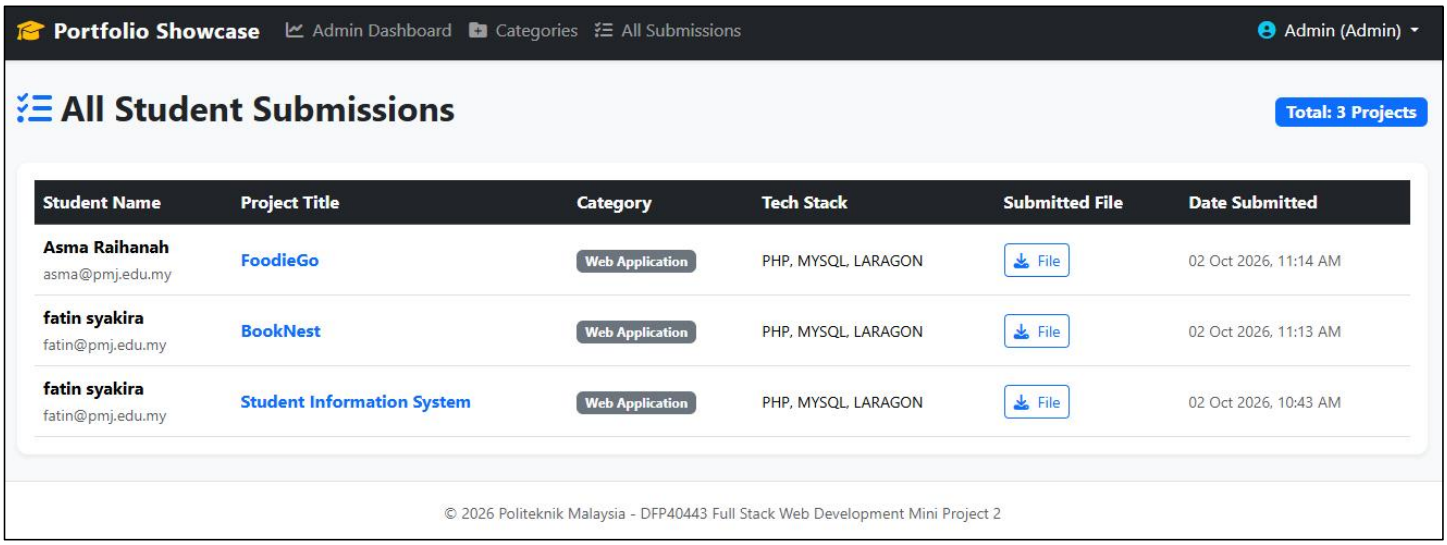


Figure 8

7. Handle all form submissions securely and ensure no invalid or empty data is stored in the database.

C : Multi-Page Application Development

1. Develop a multi-page web application using PHP.
2. The system must include the following pages:
   a. Public Pages: Login Page, Register Page
   b. Student Pages: Dashboard, Submit Assignment, View Own Submissions
   c. Admin Pages: Dashboard, Create Assignment, View All Submissions
3. Create a consistent navigation menu for Admin and Student users based on roles.
4. Use reusable components using include/require:
   a. header.php (navigation and session handling)
   b. footer.php (footer layout)
   c. db.php (database connection)
5. Ensure consistent UI design across all pages using external CSS.
6. Ensure proper page routing and redirection after login based on user roles.
7. Test and debug all pages to ensure smooth navigation and functionality.

D : AJAX Development

1. Implement AJAX using fetch API or XMLHttpRequest.

   a. AJAX must be implemented in the Student Dashboard page.

   b. Required features:

   - Live search for assignments (without page refresh)
   - Dynamic loading of assignment list
   - Display results dynamically in the dashboard

   c. Ensure smooth and responsive user experience using asynchronous data loading.

   d. Expected page shown in Figure 9-10.

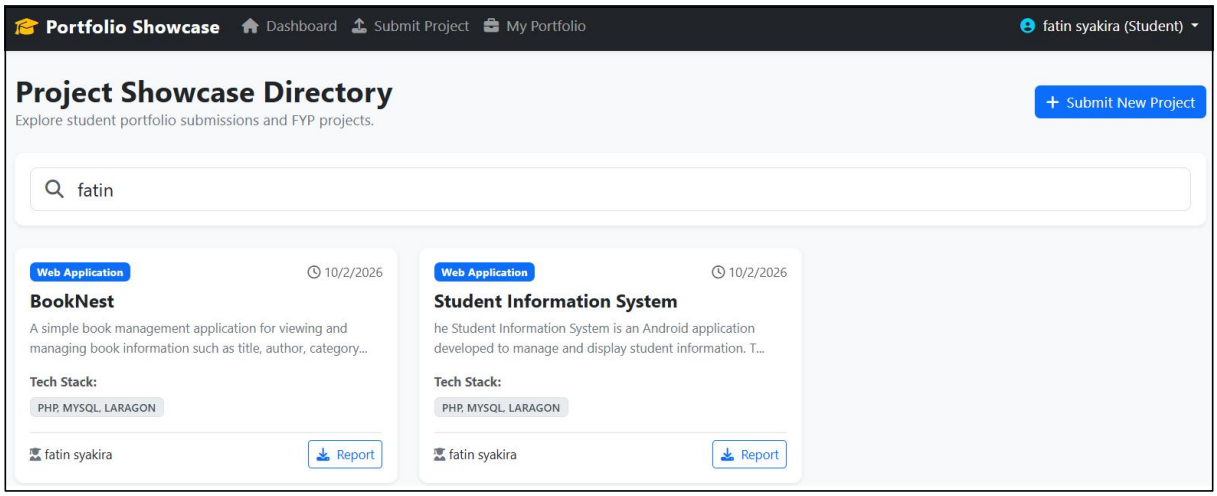


Figure 9

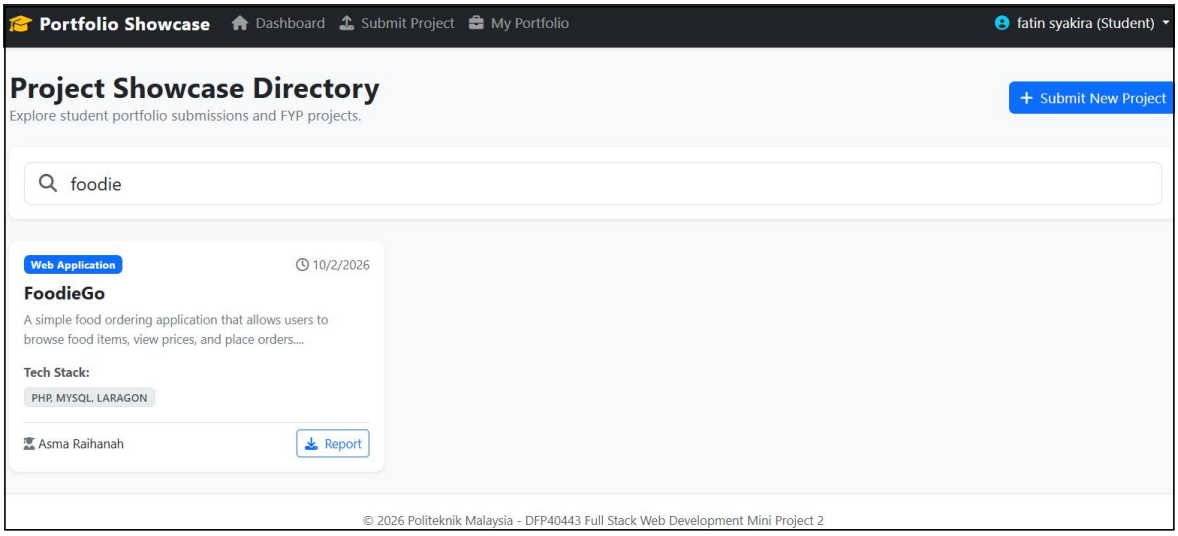


Figure 10

E : File Handling & Upload

1. File upload must be implemented in the Student Submit Assignment page.
2. Allow students to upload assignment files (e.g., PDF, DOCX, TXT).
3. Store uploaded files in a dedicated server folder (/uploads).
4. Store file path and related information in the database.
5. Admin must be able to view and download uploaded files in the View Submissions page.
6. Validate file type and file size before upload to ensure security.

F : Database Integration

1. Create a database named 'assignment_system'.
2. Create all tables based on the ERD provided in Figure 11.
3. Define Primary Keys and Foreign Keys properly.
4. Apply correct relationships:
   a. One user can submit many assignments
   b. One assignment can have many submissions
5. Use prepared statements for all database operations (SELECT, INSERT, UPDATE, DELETE).
6. Use SQL JOIN to display combined data in Admin View Submissions page (e.g., student name, assignment title, file).

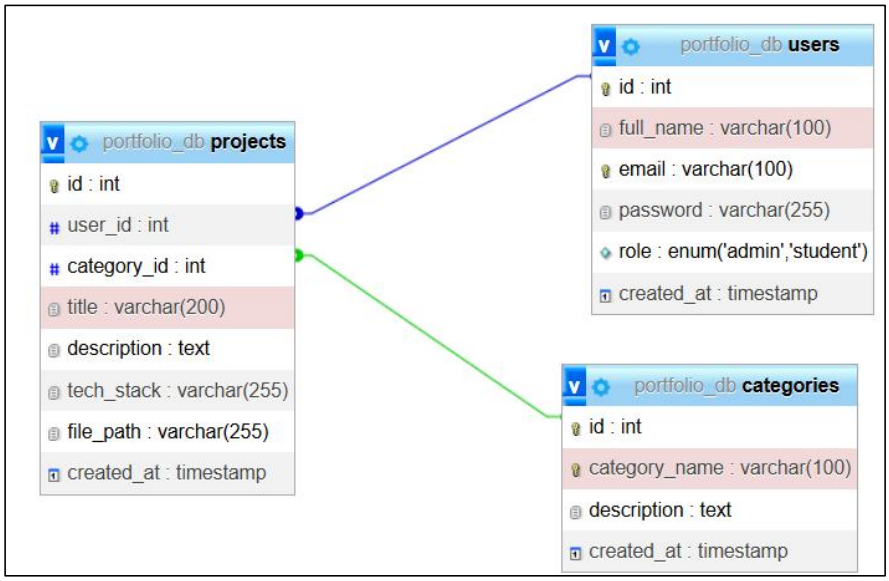


Figure 11

G : Deployment & Version Control

1. Prepare the system for deployment by organizing a clean and structured project folder.
2. Use Git for version control throughout the development process.
3. Students are required to clone the project repository and set up the project in their local environment.
4. Git must be used to manage version control during development.
5. Students are required to perform regular commits with meaningful messages and push updates to GitHub repository.
6. Ensure the final project is successfully uploaded to GitHub before submission.
7. The project must run successfully on local server (Laragon) after cloning and setup.
8. Expected page shown in Figure 12.

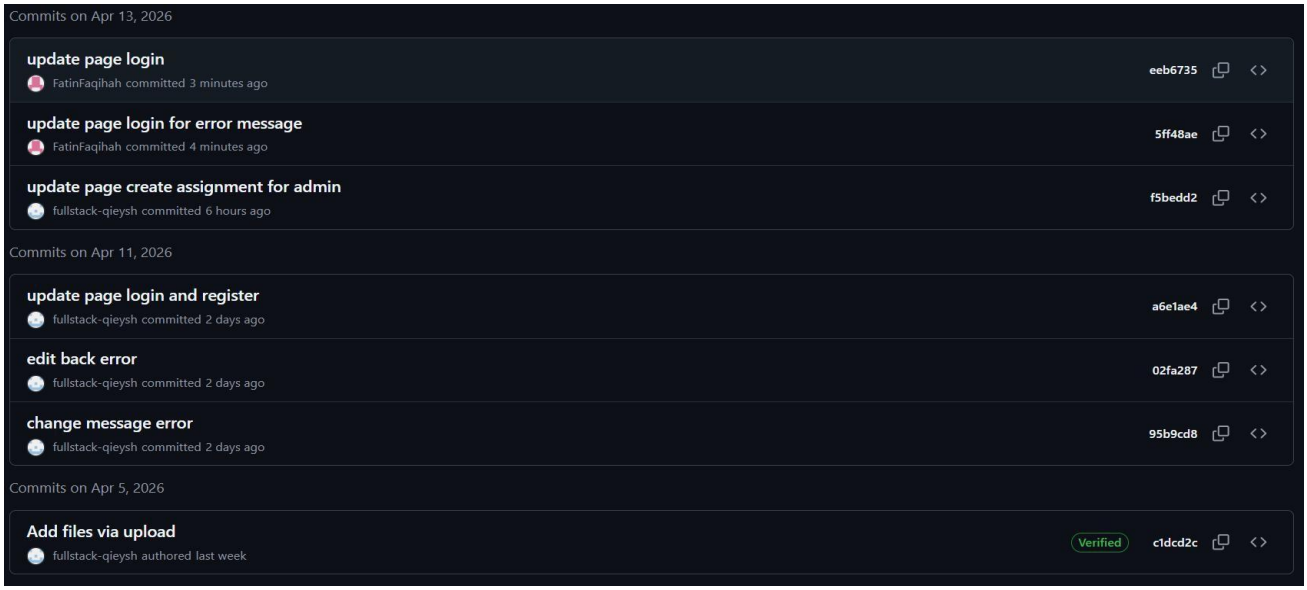


Figure 12

H : Expected Output

1. The system must have a clean and user-friendly interface.
2. The system must support role-based access (Admin and Student).
3. Admin must be able to:

* Create assignments
* View all student submissions
* Download uploaded files

4. Student must be able to:

   * Register and login

   * View assignments

   * Submit assignments with file upload

   * View own submissions

5. The system must correctly display dynamic data from the database.

Submission Requirements:

Each group must submit one (1) complete submission containing:

- Complete project folder (ZIP file)
- Database export file (.sql)
- GitHub repository link (must contain full project with commit history)
- README file including database name and setup instructions
- Report: PDF file. Content and sequence of the report:
    - o Project Description (Assignment Submission Management System)
    - o System Features (based on syllabus requirements)
    - o GitHub Repository Link
    - o Screenshot of Git Commit History (showing contribution from all members)
    - o Database name and screenshot of database structure / ERD
    - o Source Code – Typed (screenshots of code are NOT accepted)
    - o Output – Screenshots (with clear image and explanation)
- Each group member must contribute to the project (evidence will be evaluated through Git commit history).

Marking:

Marks will be awarded based on the provided rubric. Students are expected to follow the rubric criteria carefully to achieve full marks.

Name: _____ **Registration Number:** _____ **Class:** _____

RUBRIC MINI PROJECT 2

CRITERIA	4 Excellent	3 Good	2 Satisfactory	1 Weak	Weightage	Score
User Authentication & Authorization -Registration & Login -Password hashing -Role-based access -Session management	Fully functional authentication system with secure password hashing, proper role-based access, and session management	Authentication system mostly functional with minor issues in role or session handling	Basic login and registration working but lacks proper security or role implementation	Authentication implemented but contains major errors or is incomplete	$\frac{}{4}$*15 =	
Form Processing & Validation	All forms handled correctly using POST with complete client-side and server-side validation	Most validation implemented correctly with minor issues	Basic validation present but incomplete or inconsistent	Validation implemented but contains major errors or missing key components	$\frac{}{4}$*12 =	
Multi-Page Application Development -Admin & Student pages -Navigation -Reusable components -UI consistency	Complete multi-page system with proper navigation, reusable components, and consistent UI	Most pages are functional with minor navigation or UI issues	Basic pages exist but lack consistency or structure	Multi-page structure implemented but incomplete or not functioning properly	$\frac{}{4}$*12 =	
AJAX Development -AJAX implementation -Live search -Dynamic loading	AJAX fully implemented with dynamic data loading and no page refresh	AJAX works with minor issues in functionality	Basic AJAX implemented but limited features	AJAX implemented but contains errors or not functioning properly	$\frac{}{4}$*15 =	
File Handling & Upload	File upload, validation, storage, and retrieval fully functional and secure	File upload works with minor issues	Basic file upload implemented but lacks validation or retrieval	File upload implemented but contains major errors	$\frac{}{4}$*15 =	

CRITERIA	4 Excellent	3 Good	2 Satisfactory	1 Weak	Weightage	Score
Deployment & Version Control	Proper Git usage with consistent commits, meaningful messages, and contributions from all members	Git used with minor issues (limited commits or uneven contribution)	Minimal Git usage with few commits	Git implemented but lacks proper usage or evidence	$\frac{}{4}$ *15 =	
Originality of Submission	Clearly original structure and logic	Mostly original work	Standard implementation	Similar to peers / AI	$\frac{}{4}$ *8 =	
Timeliness of Submission	Submitted on time	< 1 day late	1-2 days late	More than 2 days late	$\frac{}{4}$ *8 =	
				TOTAL	**/100**	

* Zero (0) mark will be given if students do not answer or perform the criteria needed.